

Operating Multi-Node Full Fine-Tuning on NVIDIA B300:

A Field Report on Telemetry-Based Triage, Negative Results, and Operational Hardening

Seon Ho Kim¹, Ui Jeong Jeon¹, Su Hyeon Kim¹, and Min Tae Hwang¹

¹Samsung SDS, {seonho1499.kim, uijeong.jeon, sensible.kim, mr_rye.hwang}@samsung.com

August 7, 2026

Abstract

We report operational experience full-fine-tuning a 32.76B-parameter dense model (Qwen3-32B) on $16 \times$ NVIDIA B300 (two nodes, FSDP / ZeRO-3)—among the first published field accounts on this accelerator. **We claim no new algorithm.** The individual mechanisms we use are established practice; our contribution is the *integrated* field experience and a set of *calibrated measurements* on new hardware. Concretely we offer four practitioner artifacts. **(1) A B300-calibrated power-draw triage table** that distinguishes compute / communication / data-starvation / checkpoint-or-deadlock / idle by board wattage, motivated by the well-known failure of GPU-utilization% to reflect real progress (it reads 100% during an NCCL hang). **(2) A set of honest negative results** that dispel common optimization folklore at this scale: a controlled A/B in which per-step NFS reading matches a pretokenized local cache (~ 53 k tok/s) because the corpus fits in page cache and the job is compute-bound; and a reconstruction of an earlier “throughput collapse” as NFS/CPU contention rather than a storage-medium limit. **(3) Calibrated 4/8/16-GPU strong-scaling and GPU-hour numbers** on B300 (near-linear, as expected in this regime; we report absolute values as reference data). **(4) A worked failure case**—an epoch-end NCCL deadlock from per-rank token-packing imbalance—together with a 2.7-second pre-run invariant gate and an external watcher that turn multi-hour silent failures into instant rejections. We are explicit that this deadlock and its remedy correspond to PyTorch’s documented `Join` context manager and the standard `drop_last` / equalize-to-minimum practice; we position our in-situ instantiation against that prior art and report the GPU-hours the failure cost and the gate saves. The transferable takeaway is operational, not algorithmic: *for data-dependent data-parallel jobs, watch power rather than utilization, and verify invariants before launch—a passing smoke test is not evidence of a safe full run.*

1 Introduction

This is an experience report. We do not introduce a new training algorithm, a new parallelism strategy, or a new optimizer. Instead we share what it actually took to operate a multi-node full fine-tuning job on **NVIDIA B300**—brand-new hardware (Blackwell Ultra, 2025–26) for which little operational experience is yet public—and the measurements and triage practices we found valuable and believe transfer to other practitioners.

We are upfront about novelty, because it shapes how this report should be read. The building blocks we rely on already exist and are well documented: FSDP/ZeRO-3 sharding; the fact that uneven per-rank inputs desynchronize collectives (the documented motivation for PyTorch’s `Join` context manager); `drop_last` / equalize-to-minimum batch handling; checkpoint/restart fault tolerance; and the use of hardware telemetry to detect stalls. None of these is our invention. What we offer is the *integration* of these into a working operational discipline on B300, plus *calibrated numbers*—power bands, strong-scaling efficiency, data-pipeline throughput, and the GPU-hour cost of a real silent failure—that are otherwise hard to find for this hardware and workload size.

The report is organized around four practitioner-facing deliverables:

- **Telemetry-based triage** (§3): a B300-calibrated mapping from board *power* (not utilization%) to execution phase, with the flight recorder as confirmatory tool, packaged as a one-line monitor and a decision procedure.
- **Negative results** (§4): controlled measurements showing where we *wasted* effort—local-cache optimization that bought no throughput at this scale—and a correction of an earlier misdiagnosis.
- **Calibrated scaling reference** (§6): full-epoch 4/8/16-GPU throughput and GPU-hour numbers on B300, as reference data.
- **A worked failure + operational hardening** (§5, §7): an epoch-end deadlock from packing imbalance, positioned explicitly against `Join/drop_last`, plus a 2.7-second preflight gate and external watcher, with a cost accounting.

The recurring theme is operational rather than algorithmic: in distributed training, *utilization lies*, *power tells the truth*, and *a passing smoke test is not evidence of a safe full run* when per-rank workload is data-dependent.

2 Background and Experimental Setup

2.1 Full fine-tuning memory and the need for sharding

Full fine-tuning a dense model in bf16 mixed precision with Adam requires ~ 16 bytes/parameter: parameters bf16 (2) + gradients bf16 (2) + fp32 Adam master (4) + m (4) + v (4). For Qwen3-32B (32.76B params) this is **525 GB of model states**, far beyond a single GPU. We shard with **PyTorch FSDP in FULL_SHARD mode (equivalent to ZeRO-3)** across all 16 GPUs, giving 32.8 GB of model states per GPU; the measured live footprint including activations (gradient checkpointing off) is **82 GB/GPU**, $\sim 30\%$ of the B300’s 275 GB.

2.2 Cluster and software

Component	Value
GPUs	16 $\times$ NVIDIA B300 SXM6 AC (2 nodes $\times$ 8); 1100 W TGP, 275 GB HBM
Interconnect	InfiniBand (NDR) + GPUDirect RDMA; NVLink intra-node
Software	NGC container (PyTorch 2.12, transformers 5.12, native FSDP)
Storage	shared NFS (<code>/dataset</code>); per-node local NVMe cache; ~ 4 TB RAM/node
Model	Qwen3-32B (32.76B, dense)
Dataset	Nemotron-Personas-Korea, 1,000,000 records, ~ 1.01 B tok/epoch

2.3 Training configuration

Sequence length 2048; micro-batch 1; gradient accumulation 8; world size 16, giving a global batch of 128 sequences = **262,144 tokens/step**. One epoch is $\sim 3,854$ steps. AdamW, cosine warmup, lr $1e-5$, bf16 MixedPrecision, Qwen3DecoderLayer auto-wrap, gradient checkpointing **off** (memory headroom makes recomputation pure waste; off is $\sim 1.6\times$ faster—see §4.3). Checkpoints are bf16 consolidated HF-format shards written to NFS every 500 steps, plus once at epoch end.

2.4 Data pipeline

Records are sharded across ranks with `datasets.shard(world, rank)` (disjoint per rank). Each record is tokenized once and **packed into fixed 2048-token blocks** persisted as a node-local NVMe `.i32` file (~ 1.9 GB/node). We adopted packing after a production run exhibited a sustained **data-starvation collapse from 52k to 21k tok/s** (the ~ 575 W “uneven-utilization” signature of §3), driven by single-threaded tokenization plus page-cache cooling on the shared NFS under multi-tenant load. A later *controlled* measurement (§4) shows the storage *medium* itself is not the bottleneck, which localizes the residual value of packing to **determinism** (a bit-exact, resumable token stream) and **robustness to NFS/CPU contention** rather than throughput. The per-rank block count is $\lfloor \text{file_bytes}/4/2048 \rfloor$ —the quantity that turns out to be unequal across ranks and drives the failure case of §5.

3 Telemetry-Based Triage: Watch Power, Not Utilization

Our single most useful operational practice was to stop trusting GPU utilization and start reading **board power**. We make no claim that power-based diagnosis is new—vendor guides, SRE write-ups, and DCGM-based tooling all use power and related telemetry to detect stalls (§8). Our contribution here is narrow and concrete: **the B300-calibrated wattage bands**, and an integrated decision procedure that classified every stall we met without attaching a profiler.

3.1 Why utilization lies

`nvidia-smi`’s utilization percentage reports only that *a kernel is scheduled*, not that it does useful compute. NCCL collectives, memory copies, and IO waits all register as 100% “utilization” while the GPU makes little progress. The flagship symptom (§5) is that during a full NCCL deadlock utilization stays pinned at 100% even though throughput is exactly zero. Board power, by contrast, tracks real work: it falls when the silicon waits.

3.2 The B300 power-draw bands

Power (per GPU)	Utilization pattern	Phase / interpretation
$\sim 940\text{--}1040$ W	100%, uniform	compute (matmul-bound)—healthy
~ 640 W	100%, uniform	communication; if sustained, overlap failing
~ 575 W	55–100%, uneven	data starvation (stragglers)
$\sim 190\text{--}195$ W	1 rank 0% / rest 100% (deadlock) or rank 0 only 0% (checkpoint)	hang / IO (distinguish by shape)
~ 135 W	0%, all ranks	idle

Absolute wattages are hardware/firmware-specific (§9); the *ordering* and *per-rank-shape discriminators* below are what transfer.

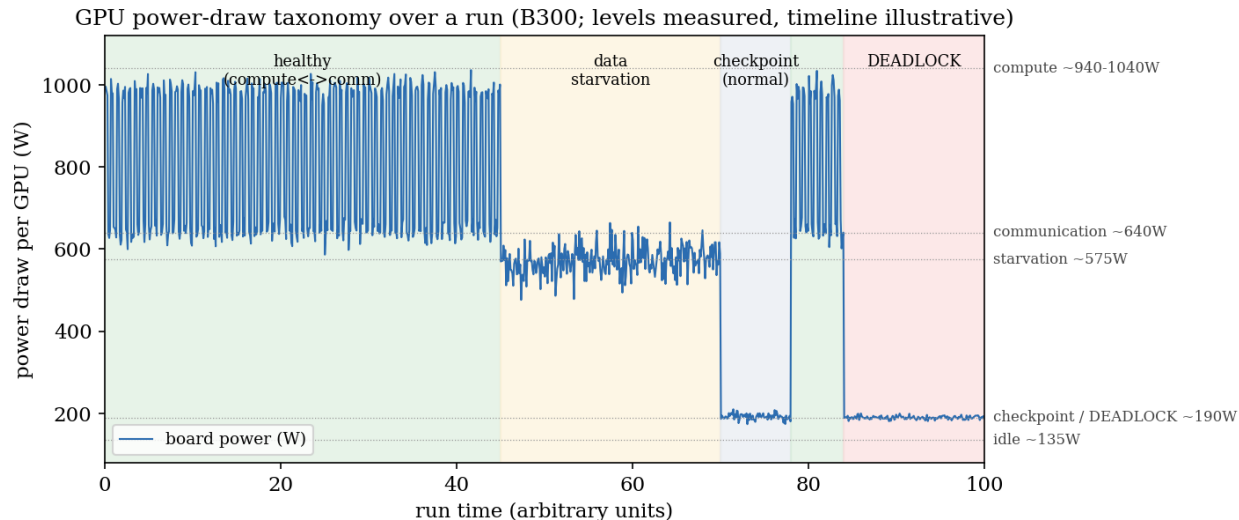


Figure 1: GPU power-draw taxonomy over a run. Power levels are measured B300 values; the timeline is an illustrative arrangement of phases. Utilization can read 100% in several of these states, so power + per-rank shape—not utilization—is the reliable signal.

3.3 Two discriminations that resolve ambiguous states

1. **Deadlock vs. healthy checkpoint** (both $\sim$ 190 W). In a *normal* checkpoint only **rank 0** drops to 0% (it gathers shards and writes $\sim$ 62 GB to NFS) while others wait, resuming within $\sim$ 2 min. In a *deadlock* the shape is asymmetric the other way—**one rank at 0% and the rest pinned at 100%** with power floored—and never recovers.
2. **Starvation vs. communication** (575 vs. 640 W). Communication keeps all ranks *uniformly* busy; starvation produces rank-to-rank utilization scatter, because the bottleneck is upstream of the GPUs.

3.4 The triage procedure

Combined, the bands and discriminators form a fast decision procedure needing only `nvidia-smi` and, for confirmation, the NCCL flight recorder:

1. Sample power + per-rank utilization:

```
nvidia-smi --query-gpu=index,utilization.gpu,power.draw,memory.used \
--format=csv -l 1
```

2. Map power to a band. $\sim$ 940–1040 W uniform $\rightarrow$ healthy; stop.
3. $\sim$ 190 W $\rightarrow$ inspect per-rank shape: rank-0-only-0% $\rightarrow$ normal checkpoint (wait $\sim$ 2 min); one-rank-0%-rest-100% persisting across two samples $\rightarrow$ suspect deadlock.
4. Confirm with the flight recorder (`TORCH_NCCL_TRACE_BUFFER_SIZE`): different collectives on different ranks is conclusive (§5).

This classified the deadlock of §5 in seconds, before we opened the flight recorder, and we wire its signature into the external watcher (§7).

4 Negative Results: Where Optimization Effort Was Wasted

Experience reports are most useful when they record what *did not* work. We present these precisely because we initially believed the opposite and spent effort accordingly.

4.1 A pretokenized local cache buys no throughput here

We A/B-tested the two data paths at 16 GPUs, 300 steps each, holding everything else fixed. **A** = shared NFS, per-step `datasets.shard` read + per-step tokenization (no packing). **B** = pretokenized, packed, node-local cache. To expose any cold penalty in A we dropped the page cache on both nodes immediately before the cold run, then re-ran the identical rows warm.

Path	tok/s (median)	vs B
A — NFS per-step read + tokenize, cold (post <code>drop_caches</code>)	52.8k	1.00×
A — NFS per-step read + tokenize, warm	53.0k	1.00×
B — pretokenized packed node-local cache	53.0k	—

All three are within $\sim 0.5\%$. The only slow step is the first one after a cache drop (33.9k tok/s—first-touch + JIT warmup), gone by step 10. The reason is a $1000\times$ headroom: the corpus is ~ 4 GB against **4 TB of per-node RAM**, so after first touch it is resident in page cache; and even a true cold miss is cheap—a 32B FSDP step takes ~ 5 s while reading 4 GB cold takes ~ 3 s at the measured floor:

Method	GB/s	Gbps	Bounds
warm page cache (RAM)	6.34	50.7	memcpy ceiling
multi-stream concurrent <code>O_DIRECT</code> (8/4 ranks)	3.56	28.5	N-rank concurrent supply
fio seq <code>O_DIRECT</code> QD32 $\times$ 4	3.54	28.4	parallel drive peak
<code>O_DIRECT</code> QD1	2.61	20.9	single-stream drive floor
cold buffered (<code>drop_caches</code> , full set)	1.31	10.5	true first-touch miss

Lesson and cost. Before measuring, we treated NFS as a suspected bottleneck and built the pretokenize-and-cache pipeline expecting a throughput win. The win was zero at this scale—standard page-cache behavior once a dataset fits in RAM, but not obvious to us in the moment. The cache is still worth keeping, for *determinism and contention-robustness* (§4.2), not speed. **This conclusion inverts once the dataset exceeds RAM**—then every epoch re-incurs cold reads and locality optimizations begin to dominate—so we report it as scale-conditional.

4.2 The “52k $\rightarrow$ 21k collapse” was contention, not the medium

This seems to contradict an earlier incident in which throughput halved from 52k to 21k tok/s mid-run. Reconciling the two localizes the real bottleneck. The collapse occurred under **single-threaded tokenization** (`TOKENIZERS_PARALLELISM=false`, CPU load average ~ 15) **and a contended, cooling shared NFS**—not the quiet, parallel-tokenized conditions of the A/B. With parallel tokenization and an uncontended fabric, even a fully cold NFS read matches the local cache. The bottleneck was therefore **tokenization-CPU throughput and NFS contention**, never the storage medium. The earlier incident is real; our initial framing of it as a “storage bottleneck” was imprecise, and we correct it here.

4.3 Gradient checkpointing was the wrong default

With 82 GB live on a 275 GB GPU, activation memory was never the constraint, so recomputation bought nothing and cost $\sim 1.6\times$ throughput (33k vs. 53k tok/s). “Always enable gradient checkpointing” is good advice when memory-bound; in a memory-rich regime it is pure waste—verify before enabling.

5 A Worked Failure Case: The Epoch-End Deadlock

We present the failure that consumed the most time, both as a concrete application of the §3 triage and as an honest case study. **Neither the failure mode nor the fix is new**—both correspond to documented PyTorch behavior (§5.4). The value is the worked B300 diagnosis and the hardening it motivated (§7).

5.1 Observed symptom

A from-pretrained run progressed normally for ~ 5 hours: loss $1.60 \rightarrow 0.74 \rightarrow \sim 0.68$, throughput ~ 53 k tok/s, all 16 GPUs at 100% utilization. At **step 3,850 of 3,857**—entering the end-of-epoch checkpoint—all 16 GPUs hung for **three hours** until the NCCL watchdog surfaced it. (This pre-fix run had no evenfix, so each rank ran toward its own block count; evenfix later pins every rank to the global-minimum 3,854 steps, §5.) No `dmesg` OOM, no segfault, no error: pure silence, with utilization still reading 100%—the exact trap that motivated §3.

5.2 Diagnosis via triage + flight recorder

Power had floored to ~ 190 W with **one rank at 0% and the rest at 100%**—the deadlock shape, not the rank-0-only checkpoint shape. The NCCL flight recorder dump was unambiguous:

- **Rank 3** had entered an `all_reduce`—the `dist.barrier()` guarding the end-of-epoch `save_checkpoint`.
- **Ranks 0–2, 4–15** (fifteen) were blocked in a `reduce_scatter`—the backward-pass gradient reduction, i.e. still training.

Two different collectives at the same wire slot; NCCL waits forever. The fabric was innocent: throughput had been a steady 53k tok/s and IB bandwidth nominal.

5.3 Root cause: per-rank packed-block imbalance

The loader shards *rows* evenly, but persona records vary in length, so the per-rank token count varies and, after packing into 2048-token blocks, the block counts differ:

metric	value
min blocks (rank 3)	30,835
max blocks (rank 11)	30,888
spread (max – min)	53

Rank 3 ran out 53 blocks (≈ 7 optimizer steps, at 8 blocks/step) before rank 11, exited the loop, and walked into the save barrier while others reduced gradients. The 53-block spread is **0.17% of the per-rank workload**—invisible to every metric except the one that mattered, and, as a corollary,

Epoch-end collective mismatch (NCCL flight-recorder)

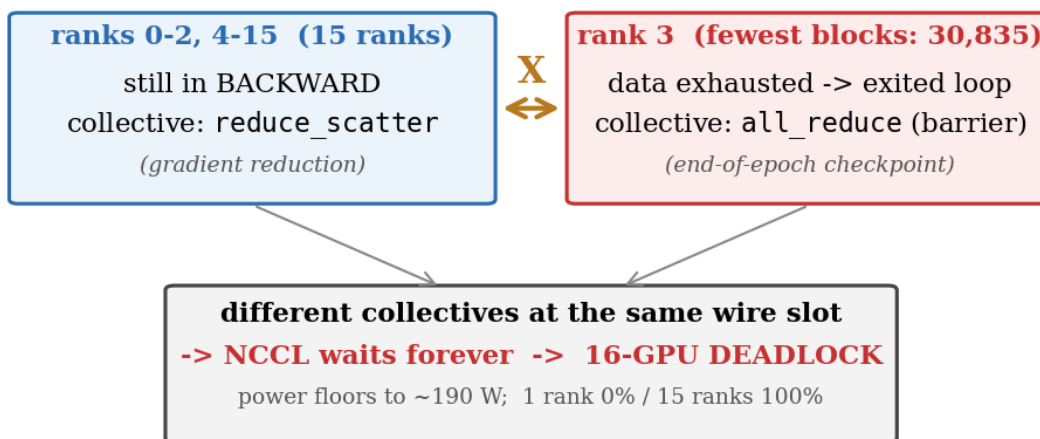


Figure 2: Epoch-end collective mismatch. The shortest-data rank (rank 3, 30,835 blocks) exits the loop one step early and enters the checkpoint `all_reduce` while the other fifteen ranks are still in the backward `reduce_scatter`. Different collectives at the same slot $\Rightarrow$ permanent NCCL deadlock.

invisible to smoke testing: a `max_steps=N` run exercises only the loop body, where all ranks are symmetric, and exits before the shortest rank exhausts its data. The hazard’s probability is zero on every prefix and one at the boundary.

5.4 Prior art: this is `Join / drop_last`, at the packing layer

The failure and remedy are documented PyTorch behavior; we are explicit rather than imply discovery:

- That **uneven per-rank inputs desynchronize collectives and hang the job** is the stated motivation for `DistributedDataParallel.join()` (since PyTorch 1.7, 2020) and the generic `torch.distributed.algorithms.Join` (1.10, 2021, covering ZeRO-style optimizers). The tutorial states it directly: “if a rank has fewer inputs, then the other ranks will hang or error.”
- The standard remedy—equalize per-rank step counts by dropping the trailing remainder—is `DataLoader(drop_last=True) / DistributedSampler`; HF Accelerate ships `even_batches` and a `join_uneven_inputs` wrapper for exactly this.
- Per-rank step imbalance from **token packing** is itself an active topic (SlimPack; Hierarchical Balance Packing), framed there as efficiency.

We considered using `Join` directly to avoid dropping any data, but it is **unavailable in our setting on two counts**. First, neither FSDP1 (`FullyShardedDataParallel`) nor FSDP2 (`fully_shard`) implements `Joinable`—only DDP and `ZeroRedundancyOptimizer` do—so an FSDP-wrapped model cannot be passed into `with Join(...)`. Second, `Join` shadows only the collectives issued *inside* a registered `Joinable`’s hooks (DDP’s gradient all-reduce, ZeRO’s optimizer collectives), not arbitrary

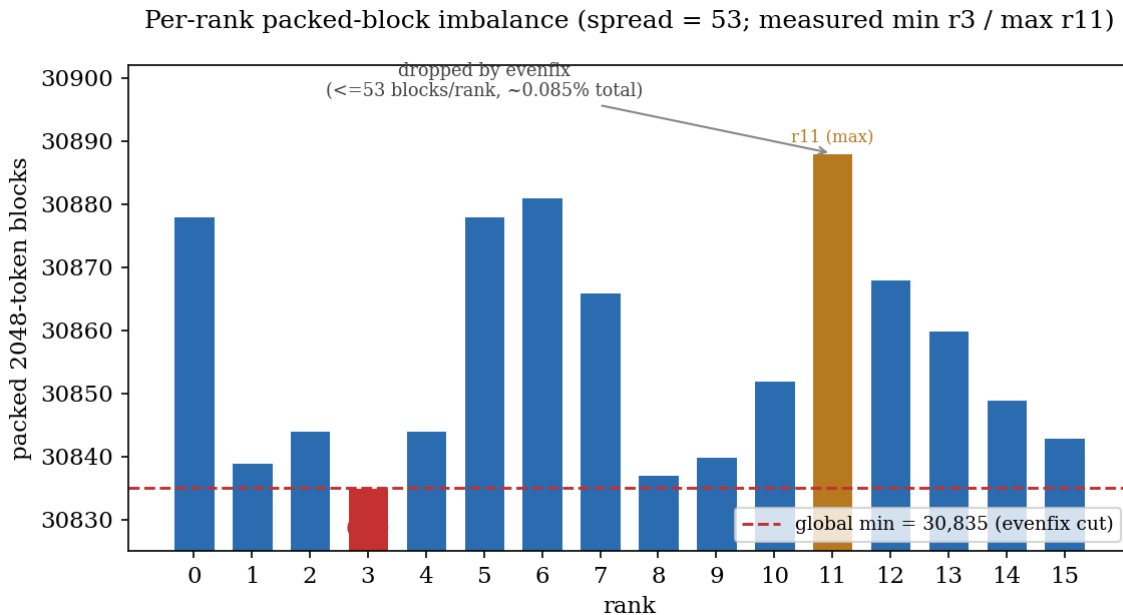


Figure 3: Per-rank packed-block counts. Even row-sharding yields unequal block counts (spread 53); the global minimum is the evenfix cut. The discarded tail is ≤ 53 blocks/rank, $\sim 0.085\%$ of tokens.

user collectives; our deadlock was at a manual end-of-epoch checkpoint `dist.barrier()`, outside that scope, which would desync even with `Join` active. For FSDP with a manual save barrier the documented path is exactly to **equalize per-rank step counts**—`drop_last` upstream, or an explicit `all_reduce(MIN)` of the iteration count; we implement the latter.¹ Note also that `drop_last` only removes a rank’s *partial* trailing batch; with micro-batch 1 over packed blocks there is no partial batch, so `drop_last` is inert and the cross-rank `all_reduce(MIN)` is what actually enforces equality.

Our “evenfix” is functionally the equalize-to-minimum pattern applied at the **post-packing block layer** rather than the sampler—a placement choice, not a new algorithm. We reframe the imbalance as a *correctness* (deadlock) hazard and contribute the worked B300 diagnosis and the preventive discipline of §7. That is the honest extent of the delta.

5.5 evenfix — the runtime guard we deployed

Before the loop, each rank reduces its local block count to the global minimum:

```
local_blocks = file_bytes // 4 // seq_len
min_blocks = dist.all_reduce(torch.tensor(local_blocks), op=ReduceOp.MIN)
# in the loop:
if blk_idx >= min_blocks:
    break # every rank stops at exactly the same step
```

All ranks now run an identical number of iterations, so the terminal save-barrier transition is symmetric and the deadlock is structurally impossible. The cost is each rank’s trailing local – min blocks—at most 53 on any single rank, $\sim 0.085\%$ of all tokens in aggregate—negligible. A live log

¹https://docs.pytorch.org/tutorials/advanced/generic_join.html; https://docs.pytorch.org/docs/stable/distributed_algorithms_join.html. Only DDP and ZeroRedundancyOptimizer are Joinable; the FSDP sources register no `Join` hook.

line confirms it: `[evenfix] local=30,849 → min=30,835`. (A pad-to-maximum variant would retain all data via duplicate blocks; we chose truncation for simplicity given the negligible loss.)

6 Calibrated Strong-Scaling Reference Numbers (B300)

We report 4/8/16-GPU scaling as **reference data**, not a finding: near-linear strong scaling for a compute-bound model at this size is the expected regime, and competitive B300 figures already exist publicly (e.g. MLPerf Training v5.1/v6.0, §8). What is useful is the *calibrated absolute numbers*. Each point is a **complete measured epoch** (4 GPUs 15,420 steps, 8 GPUs 7,710, 16 GPUs 3,854; medians over $n = 1,538/767/385$).

GPUs	Nodes	tok/s	tok/s/GPU	1 epoch (h)	GPU·h	Strong-scaling eff.
4	0.5	13.4k	3,350	20.9	83.7	100%
8	1	26.7k	3,338	10.5	84.1	100%
16	2 (IB)	53.0k	3,313	5.3	84.7	99%

Per-GPU throughput is **flat within** $\sim 1\%$ (3,313–3,350) and GPU·h per epoch is conserved at ~ 84 (4-GPU baseline; the only loss is a $\sim 1\%$ tax at 16 GPUs from the two-node IB boundary—4/8 GPUs are NVLink-only). Final loss was preserved across all three (0.674/0.673/0.67), confirming scaling does not perturb convergence. The conservation is the positive control for the §3 taxonomy: GPUs sat at ~ 940 – 1040 W (compute regime) throughout, confirming communication was hidden behind compute.

Implication for in-network aggregation (SHARP). The same $\sim 1\%$ two-node tax bounds the upside of switch-based collective offload. NVIDIA SHARP performs the gradient reduction inside the InfiniBand fabric, attacking *communication* cost; but at 16 GPUs the reduction is already overlapped behind backward compute (99% strong-scaling efficiency, GPUs at 940–1040 W), so the *exposed* communication SHARP could remove is at most the $\sim 1\%$ scaling gap—and only the communication fraction of that. By Amdahl’s law the throughput ceiling is therefore $< 1\%$ in this regime. We did not provision SHARP on this fabric (the aggregation manager grants a zero resource quota—`osts:0`, `max_groups:0`—and `sharp_coll_test` blocks at group allocation), and the expected gain is correspondingly marginal here. This is a property of the operating point, not of SHARP: its value grows at larger scale, where ring/tree all-reduce latency becomes exposed and the $O(\log N)$ in-network tree dominates. Our reference numbers should not be read as evidence against SHARP there.

The clean from-scratch epoch that **validated the §5 fix** is the 16-GPU row: 3,854 steps, `rc = 0` on the first attempt, final loss 0.67, median 53.0k tok/s. The preflight gate passed (`min_blocks = 30,835`), `evenfix` logged `local=30,849 → min=30,835`, and the epoch-end save completed with **zero watchdog, abort, or deadlock traces**—the transition that previously hung for three hours now passes cleanly. The 62 GB checkpoint loads and generates on-format, fluent in-domain text.

7 Operational Hardening and Cost Accounting

7.1 The cost of one silent failure

A single occurrence of the §5 deadlock cost, conservatively:

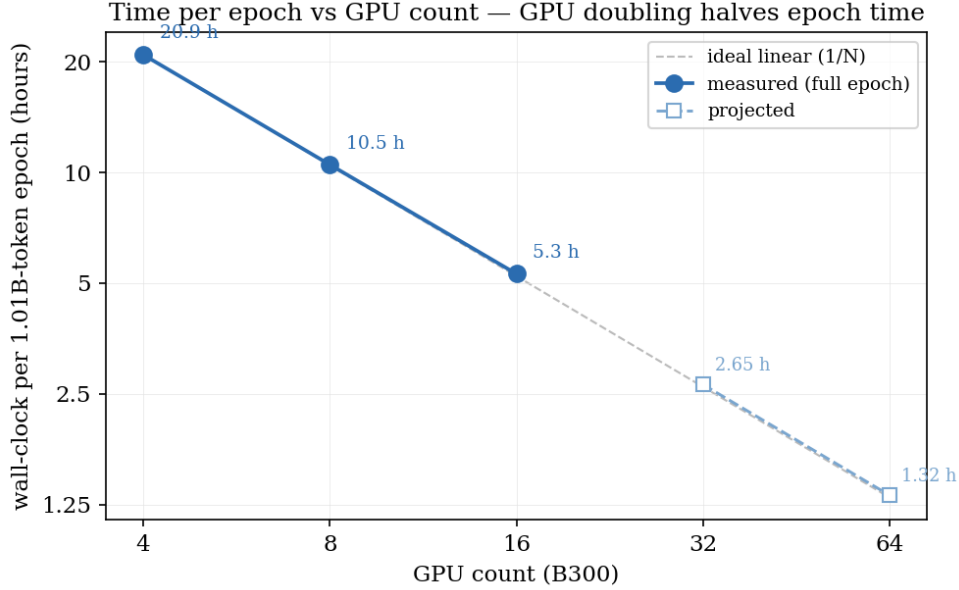


Figure 4: Wall-clock per 1.01B-token epoch vs. GPU count. Measured at 4/8/16; 32/64 projected. GPU doubling halves epoch time.

Component	GPU·h
The hang itself (16 GPU × ~3 h before watchdog)	~48
Unrecoverable progress (pre-fix epoch-end-only save; ~5 h × 16 GPU)	~85
Total per incident	~130

That is **~1.5 epochs of compute** (~84 GPU·h/epoch) thrown away by a sub-percent, sub-visible imbalance. Separately, the §4.2 starvation incident cost an estimated **~ + 3 h wall-clock** on a 5.3 h epoch ($\approx + 50$ GPU·h) before diagnosis—now caught in a single `nvidia-smi` sample.

7.2 The preflight gate — block the launch, not the run

Because “smoke pass $\neq$ full safe,” we verify *before* the expensive launch. A gate checks six invariants and refuses to start an unsafe run; its pass log doubles as a recorded clean-run pre-validation artifact.

Measured wall-clock to run all six checks across both nodes: 2.71 s.

#	Check	Failure it prevents
1	Cache integrity + per-rank block balance	missing shards; records evenfix precondition.
2	Token-total sanity ($\sim 1.01\text{B} \pm 10\%$)	partial cache build
3	GPU occupancy = 0 on both nodes	zombie/competing processes
4	<code>master_port</code> free	stale rendezvous
5	Checkpoint disk headroom (≥ 150 GB)	save-stage failure
6	(optional) IB bus-bandwidth probe	fabric degradation (opt-in)

A nonzero block spread is **WARN**, not **FAIL** (evenfix handles it); the gate records `global_min_blocks`. It is **hard-wired into the launcher**, so every full run is admitted only after passing. The economics are stark: a **2.71-second** check stands between submission and a failure that, when it slips through, costs **~130 GPU·h**.

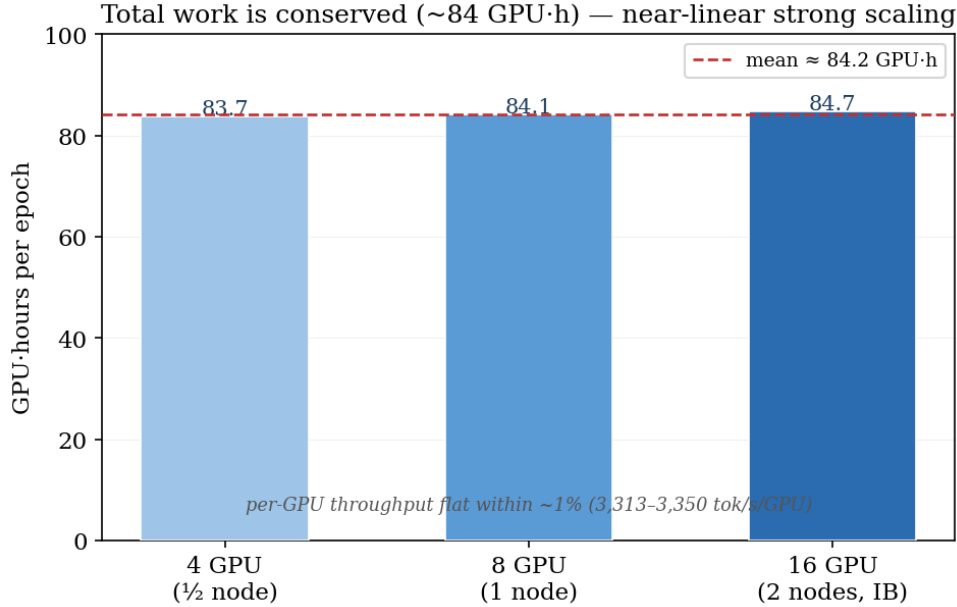


Figure 5: GPU-hours per epoch conserved at ~ 84 across 4/8/16 GPUs (99–100% efficiency, 4-GPU baseline): total work preserved, no communication bottleneck across the two-node IB boundary.

7.3 The external watcher — observe from outside the run

Nodes that deadlock cannot report their own death. A **session-independent watcher** on a separate host detects completion (epoch checkpoint appears), crash (process gone, no checkpoint), **hang** (step counter stalls, or the §3 power/utilization deadlock signature persists across two samples), and throughput floor—then emits an out-of-band alert. Recoverability was addressed orthogonally with periodic checkpointing (`save_every=500`, ~ 42 min), weights-only resume, an increased watchdog timeout (10 $\rightarrow$ 30 min), and launcher auto-retry—standard practice mentioned for completeness.

8 Related Work

Uneven inputs and packing balance. That uneven per-rank inputs cause collective desync and hangs is the documented rationale for PyTorch’s `DistributedDataParallel.join()` (1.7) and the generic `Join` (1.10), and is mitigated by `DataLoader(drop_last=True)` / `DistributedSampler` and HF Accelerate’s `even_batches` / `join_uneven_inputs`. Notably, `Join` is implemented only for DDP and ZeRO, **not FSDP** (§5.4), so FSDP users with uneven per-rank inputs must equalize step counts manually. Per-rank imbalance from token packing is active research (SlimPack, arXiv:2509.26246; Hierarchical Balance Packing, arXiv:2503.07680), framed as efficiency; we reframe it as a correctness hazard but claim no algorithmic novelty.

Telemetry-based stall diagnosis. Using GPU power and hardware telemetry to detect stalls/stragglers—including the “100% utilization during an NCCL hang” symptom—is established in vendor/SRE practice (NVIDIA DCGM; Megatron/NeMo straggler detection; troubleshooting playbooks) and in research on telemetry-based anomaly detection (arXiv:2510.26008; Mycroft, arXiv:2509.03018). Concurrent 2026 work extends telemetry-based failure detection further: observability-aware early warning that models monitoring-pipeline degradation itself (arXiv:2603.28781), and performance-counter-based stress estimation beyond utilization%

(arXiv:2511.05067). Phase-resolved power, including a named “execution-idle” state, has been characterized (arXiv:2604.04745; ASPLOS’24 LLM power characterization). We contribute B300-calibrated absolute bands, not a new principle.

Scaling/reliability at scale. Large-scale training experience reports—the OPT-175B logbook, BLOOM, MegaScale, the Llama 3 herd reliability section, Imbue’s bare-metal-to-70B account—document emergent failures at thousands of GPUs. Ours is modest scale (16 GPUs) with a documented root cause; we position it as a focused field note. Closest in genre is a concurrent operational report on a 504-GPU *B200* production cluster (arXiv:2605.09370), which analyzes pre-training failures via a multi-layer Prometheus metric suite (~ 751 metrics at 30 s granularity) and recovery workflow; it does not map board power to execution phase, and its workload is MoE pre-training rather than dense full fine-tuning. Our contributions are therefore complementary: single-command power-band triage that needs no metric infrastructure, and a full-fine-tuning-specific packing-imbalance hazard. Public B300 scaling references include MLPerf Training v5.1/v6.0.

Fault tolerance. ZeRO/FSDP sharding, overlapped communication, and checkpoint/restart are well established; our use is standard.

9 Threats to Validity and Scope

- **Scale.** We measured to 16 GPUs / 2 nodes. The deadlock mechanism is scale-independent, but the efficiency numbers (§6) should not be extrapolated past the two-node IB boundary, and at-scale reliability lessons do not follow from a 16-GPU study.
- **Single model/dataset.** The generality argument is structural, not empirical across models.
- **Power thresholds are hardware/firmware-specific.** The §3 wattages are B300 values; the taxonomy and shape discriminators should transfer, absolute thresholds must be re-calibrated.
- **Data-vs-RAM assumption.** The §4 conclusion holds only while the dataset fits in page cache; it inverts for datasets $>$ RAM.
- **Reconstructed cost.** The §7 starvation cost is estimated from logs, not a controlled measurement.

10 Conclusion and Artifact Availability

We shared operational experience full-fine-tuning Qwen3-32B on $16 \times$ B300. Rather than a new algorithm, we offer four practitioner artifacts: a B300-calibrated power-draw triage procedure (utilization lies; power tells the truth); honest negative results showing local-cache optimization buys no throughput when the dataset fits in RAM and that an earlier “storage collapse” was really CPU/NFS contention; calibrated 4/8/16-GPU strong-scaling reference numbers; and a worked epoch-end deadlock—explicitly the documented `Join/drop_last` failure, applied at the packing layer—hardened by a 2.71-second preflight gate and an external watcher that convert a ~ 130 GPU·h silent failure into an instant rejection. The transferable takeaways are operational: *watch power*, *not utilization*, and *verify invariants before launch*—a *passing smoke test is not evidence of a safe full run*.

Artifacts. The FSDP trainer (with evenfix), the preflight gate, the external watcher, the pretokenizer/packer, and the scaling/benchmark scripts are available from the authors upon reasonable request.

A Reproduction sketch

1. Pretokenize and pack the corpus into per-rank `.i32` block files (`pretokenize_persona.py`); record per-rank block counts.
2. Run the gate (`preflight_gate.sh`, `WORLD=16` `NPROC=8` `NODES=2`); confirm checks 1–5 pass (~ 3 s) and note `global_min_blocks`.
3. Launch (`full_ft_launch.sh`); the launcher invokes the gate, then `torchrun` across both nodes with `evenfix` active.
4. Monitor with the one-line power query (§3.4) and/or the external watcher; expect ~ 940 – 1040 W compute and a clean ~ 190 W rank-0-only checkpoint at epoch end.

B Key measured quantities

Quantity	Value
Model	Qwen3-32B, 32.76B dense
World	16 GPU (2×8 B300), FSDP FULL_SHARD / ZeRO-3, bf16
Tokens/step	262,144
Steps/epoch	$\sim 3,854$
Tokens/epoch	~ 1.01 B (after packing)
Per-rank blocks	min 30,835 / max 30,888 / spread 53
Tokens dropped by evenfix	$\sim 0.085\%$
Throughput (16/8/4 GPU)	53.0k / 26.7k / 13.4k tok/s
Memory	82 GB/GPU (grad-ckpt off), 30% of 275 GB
Loss	$1.60 \rightarrow 0.67$ (1 epoch)
Epoch wall-clock (16 GPU)	~ 5.3 h
GPU·h/epoch	~ 84 (conserved across 4/8/16)
Data A/B (NFS cold / warm / local)	52.8k / 53.0k / 53.0k tok/s
Cold buffered storage read	1.31 GB/s
Preflight gate wall-clock	2.71 s (6 checks, 2 nodes)
Hang duration (pre-fix)	~ 3 h before watchdog
Cost per silent failure	~ 130 GPU·h (~ 1.5 epochs)